

Revisão para a P1

Projeto, Implementação e Teste de Software

Da especificação aos casos de teste, do código aos caminhos.



Prof. Dr. João Choma

UniCesumar — Engenharia de Software

github.com/JoaoChoma

Fundamentos, artefatos, teste estrutural e planejamento

Mapa da revisão



Fundamentos e caixa-preta

Relacionar defeito, erro e falha; distinguir as perspectivas funcional e estrutural; selecionar entradas informativas.

Artefatos de teste

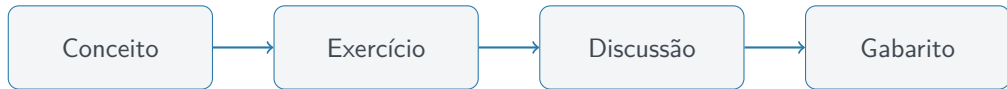
Elaborar casos de teste e procedimentos claros, completos e rastreáveis.

Caixa-branca, CFG e McCabe

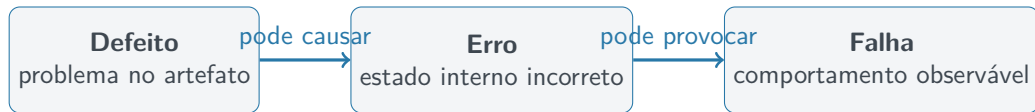
Transformar código em grafo, contar decisões, calcular complexidade e propor caminhos.

Unidade e planejamento

Explicar bons testes unitários e construir um plano de teste enxuto.



Fundamentos



Exemplo

Um operador incorreto no código é um **defeito**; durante a execução, ele produz um valor interno errado; quando o usuário recebe um desconto incorreto, observa-se uma **falha**.



Caixa-preta — “O que faz?”

- parte de requisitos e especificações;
- relaciona entradas e saídas;
- procura comportamentos incorretos;
- não depende de examinar o código.

Caixa-branca — “Como faz?”

- parte da estrutura do código;
- observa decisões e fluxo de controle;
- mede cobertura estrutural;
- exercita caminhos de execução.

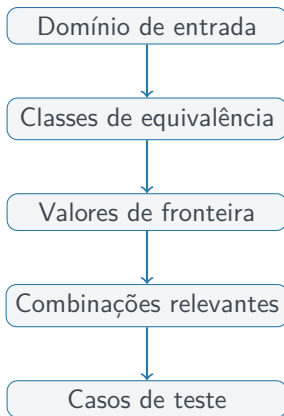
São perspectivas complementares, não substitutas.



- 1 Um valor incorreto armazenado em memória é defeito, erro ou falha?
- 2 Uma mensagem errada mostrada ao cliente é defeito, erro ou falha?
- 3 Qual abordagem deriva testes diretamente dos requisitos?
- 4 Qual abordagem permite avaliar cobertura de decisões?
- 5 Por que uma abordagem não substitui a outra?

Antes de avançar, explique cada resposta usando um exemplo concreto.

Teste funcional





Uma loja concede desconto quando:

$$\text{valorCompra} \geq 200 \quad \text{E} \quad \text{clienteVip} = \text{true}$$

Tarefa

Proponha entradas que testem a fronteira do valor e uma combinação relevante do perfil VIP.

- Onde estão as classes válidas e inválidas?
- Quais valores ficam imediatamente abaixo, na fronteira e imediatamente acima?
- O que muda quando o cliente não é VIP?



Caso	Valor	VIP	Por quê?
CT01	R\$ 199,99	true	imediatamente abaixo
CT02	R\$ 200,00	true	valor de fronteira
CT03	R\$ 200,01	true	imediatamente acima
CT04	R\$ 200,00	false	combinação relevante

Pergunta decisiva

Por que R\$ 500,00 com VIP verdadeiro provavelmente acrescenta menos informação que R\$ 200,00 com VIP verdadeiro?

Porque o segundo caso exercita exatamente o ponto em que o comportamento deve mudar.



Equivalência

- divide entradas com comportamento esperado semelhante;
- escolhe representantes.

Fronteira

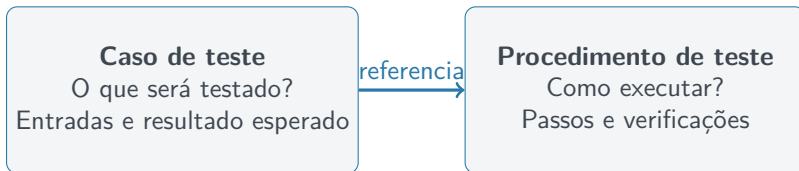
- testa limites;
- verifica os dois lados e o próprio limite.

Combinação

- cruza condições relevantes;
- revela interação entre entradas.

O melhor caso não é o mais extremo; é o que mais informa.

Artefatos de teste



- O caso define a condição que precisa ser verificada.
- O procedimento torna a execução repetível por outra pessoa.
- A identificação entre os artefatos garante rastreabilidade.



Campo	Conteúdo
Identificação	CT_LOGIN_01
Objetivo	Validar autenticação com credenciais válidas.
Pré-condição	Usuário “joao” cadastrado e ativo.
Entradas	login = joao; senha = 123456
Resultado esperado	Usuário autenticado e direcionado à página inicial.
Ambiente	Banco e aplicação de teste disponíveis.
Procedimento	PT_LOGIN_01



Identificação: PT_LOGIN_01

Caso: CT_LOGIN_01

- 1 Abrir o sistema.
- 2 Acessar a tela de login.
- 3 Informar “joao”.
- 4 Informar “123456”.
- 5 Clicar em *Entrar*.
- 6 Verificar se a página inicial foi apresentada.

Critério de qualidade

Uma pessoa que não escreveu o caso deve conseguir executar o procedimento e chegar à mesma verificação.



Requisito

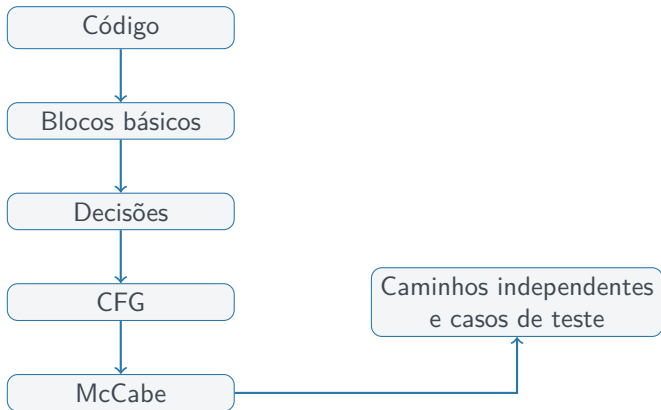
O sistema deve permitir cadastrar um cliente com nome, e-mail válido e CPF ainda não registrado.

Produza:

- 1 um caso de teste com identificação, objetivo, pré-condição, entradas e resultado esperado;
- 2 um procedimento com identificação, referência ao caso, passos e verificações.

Pergunta de conferência: o caso diz **o que** verificar e o procedimento explica **como** executar?

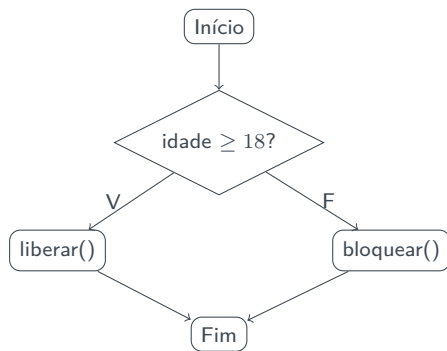
Teste estrutural





```
if (idade >= 18) {  
    liberar();  
} else {  
    bloquear();  
}
```

- Decisões: 1
- Caminhos básicos: 2
- $V(G) = 1 + 1 = 2$





```
for (int i = 0; i < 10; i++) {  
    if (valor[i] > 0) {  
        total++;  
    }  
}
```

Decisões

- condição do for;
- condição do if.

Complexidade

$$D = 2 \quad V(G) = D + 1 = 3$$

O retorno do laço precisa aparecer no CFG.

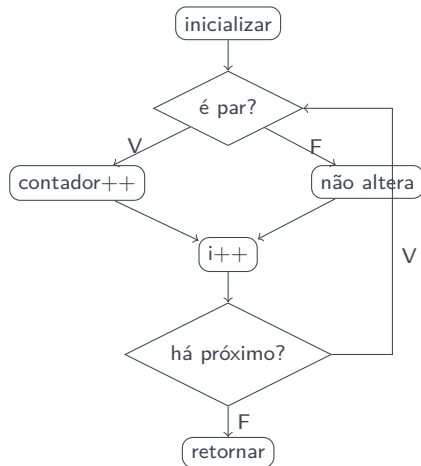


```
int contarPares(int[] numeros) {  
    int contador = 0;  
    int i = 0;  
    do {  
        if (numeros[i] % 2 == 0) {  
            contador++;  
        }  
        i++;  
    } while (i < numeros.length);  
    return contador;  
}
```

- 1 Identifique blocos e decisões.
- 2 Desenhe o CFG e identifique o ciclo.
- 3 Conte nós N e arestas E .
- 4 Calcule $V(G)$.
- 5 Identifique caminhos independentes.
- 6 Proponha entradas para exercitá-los.

- D1: número atual é par?
- D2: ainda existem elementos?

$$V(G) = D + 1 = 2 + 1 = 3$$





Pelo grafo

$$V(G) = E - N + 2$$

E é o número de arestas e N é o número de nós, para um grafo conectado.

Por estruturas simples

$$V(G) = D + 1$$

D é o número de decisões. Esta forma também confere se o grafo foi construído corretamente.

A complexidade indica o tamanho de uma base de caminhos independentes.



```
if (x > 0) { a(); }  
if (y > 0) { b(); }
```

Combinações possíveis

- V, V
- V, F
- F, V
- F, F

Complexidade ciclomática

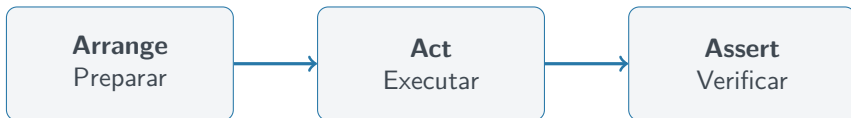
$$D = 2 \Rightarrow V(G) = 3$$

Procuramos uma base de caminhos **linearmente independentes**, não necessariamente todas as combinações.



- 1 Separe sequências lineares em blocos básicos.
- 2 Transforme cada condição em um nó de decisão.
- 3 Represente saídas verdadeira e falsa.
- 4 Mostre explicitamente o retorno dos laços.
- 5 Una ramos quando o fluxo volta a ser comum.
- 6 Conte nós e arestas.
- 7 Calcule $V(G) = E - N + 2$.
- 8 Confira com $V(G) = D + 1$.
- 9 Se os valores divergirem, revise o grafo.

Teste unitário e planejamento



Ideia central

A unidade deve ser exercitada com entradas conhecidas e o resultado observado deve ser comparado com um resultado esperado conhecido.



```
@Test
void deveSomarDoisValores() {
    // Arrange
    Calculadora calc = new Calculadora();

    // Act
    int resultado = calc.somar(2, 3);

    // Assert
    assertEquals(5, resultado);
}
```

O nome descreve o comportamento; a asserção torna o resultado verificável.



- independente de outros testes;
- determinístico;
- isolado da infraestrutura externa;
- rápido e repetível;
- baseado em entradas conhecidas.
- resultado esperado conhecido;
- cobertura adequada ao risco;
- comportamentos e exceções relevantes;
- estados do objeto, quando aplicável;
- falha que explica o problema.



Pergunta	Elemento do plano
Por quê?	Objetivo mensurável
Como?	Estratégia ou abordagem
Quem?	Responsabilidades
Onde?	Ambiente de teste
O quê?	Itens a testar
Quando?	Cronograma ou agenda
Fazer o quê?	Tarefas de teste



Em grupos, elabore um plano de teste de no máximo uma página para uma entrega incremental do módulo **Cadastro de Cliente**.

- 1 Defina um objetivo mensurável.
- 2 Escolha a estratégia de teste.
- 3 Atribua responsabilidades.
- 4 Descreva o ambiente.
- 5 Liste os itens cobertos.
- 6 Organize o cronograma.
- 7 Declare as tarefas de preparação, execução e encerramento.



Elemento	Exemplo
Objetivo	Validar 100% dos critérios de aceite do cadastro antes da demonstração da sprint.
Estratégia	Testes funcionais de equivalência e fronteira; testes unitários das validações; reteste de defeitos.
Responsáveis	Pessoa desenvolvedora: unitários e correções; QA: casos, execução e relatório; PO: aceite.
Ambiente	Aplicação de homologação, navegador homologado e banco com massa controlada.
Itens	Inclusão, campos obrigatórios, e-mail, CPF duplicado e mensagens de validação.
Cronograma	Dia 1: preparação; dia 2: execução; dia 3: correções e reteste; dia 4: aceite.
Tarefas	Preparar dados, revisar casos, executar, registrar evidências, reportar defeitos e consolidar resultado.

Desafio integrado



```
public double calcularDesconto(  
    double valor, boolean vip, int quantidade) {  
    double desconto = 0;  
    if (valor >= 200) {  
        desconto = 10;  
    }  
    if (vip) {  
        desconto += 5;  
    }  
    if (quantidade >= 10) {  
        desconto += 5;  
    }  
    return desconto;  
}
```



Sem usar a estrutura interna como ponto de partida:

- 1 identifique as classes de equivalência de cada entrada;
- 2 identifique as fronteiras de valor e quantidade;
- 3 selecione combinações relevantes envolvendo vip;
- 4 proponha um conjunto enxuto de casos de teste;
- 5 informe o resultado esperado de cada caso.

Pergunta de controle

Cada caso acrescenta informação ou repete uma condição já coberta?

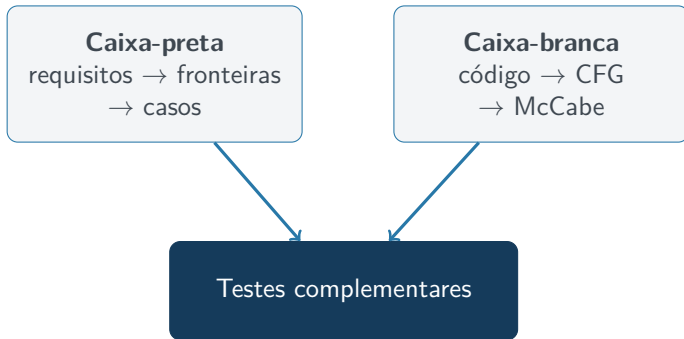


Partindo da estrutura interna:

- 1 identifique as decisões;
- 2 separe os blocos básicos;
- 3 desenhe o CFG;
- 4 conte N e E ;
- 5 calcule $V(G)$ de duas formas;
- 6 proponha uma base de caminhos independentes;
- 7 escolha entradas capazes de exercitar esses caminhos.

Conferência

O grafo mostra três decisões e todos os ramos podem ser associados a entradas concretas?



- A caixa-preta revela problemas no comportamento esperado e nas interfaces.
- A caixa-branca revela lacunas na execução de ramos e caminhos internos.
- Casos, procedimentos e plano tornam a estratégia executável e rastreável.



- Explico defeito $\rightarrow$ erro $\rightarrow$ falha com exemplo?
- Distingo caixa-preta e caixa-branca?
- Selecciono classes, fronteiras e combinações relevantes?
- Elaboro um caso de teste completo?
- Escrevo um procedimento reproduzível?
- Transformo decisões e laços em CFG?
- Calculo e confiro $V(G)$?
- Distingo caminhos independentes de todas as combinações?
- Explico um teste unitário eficaz?
- Estruturo um plano enxuto em sete elementos?

Se consigo explicar, representar e aplicar, estou preparado.